SOFTWARE ENGINEERING

LAB 3

Component Modelling & Architectural Pattern Selection

Problem Statement #32
Personal Subscription & Recurring Expense Auditor

Name	Sai Rishi Gangarapu
SRN	PES1UG24CS400
Course	Software Engineering
Lab	Lab 3 – Component Modelling & Architectural Pattern Selection
Reference Product	Rocket Money-style personal finance / subscription auditor

Prepared from the Lab 1 requirements and use-case model supplied for Problem Statement #32.

1. System Overview and Scope

The Personal Subscription & Recurring Expense Auditor is a Rocket Money-style personal finance application focused on automatically identifying recurring expenses and subscriptions from transaction data. The system connects to users' bank accounts through secure read-only integrations, analyzes transaction history, presents subscription and monthly burn-rate information, alerts users before renewals, and provides actionable cancellation guidance. It also supports anonymized aggregate reporting for a Finance Auditor.

Primary system goals

- Automatically detect recurring billing patterns from transaction logs.
- Import transaction history through secure read-only banking integrations.
- Provide a subscription dashboard and projected monthly recurring expenditure.
- Alert users at least 72 hours before identified subscription renewals.
- Provide cancellation guides or links, with an optional concierge cancellation extension.
- Generate anonymized, aggregated monthly burn-rate reports in CSV and PDF formats.

2. Architectural Pattern Selected

Selected pattern: Layered Architecture with Modular Component Decomposition

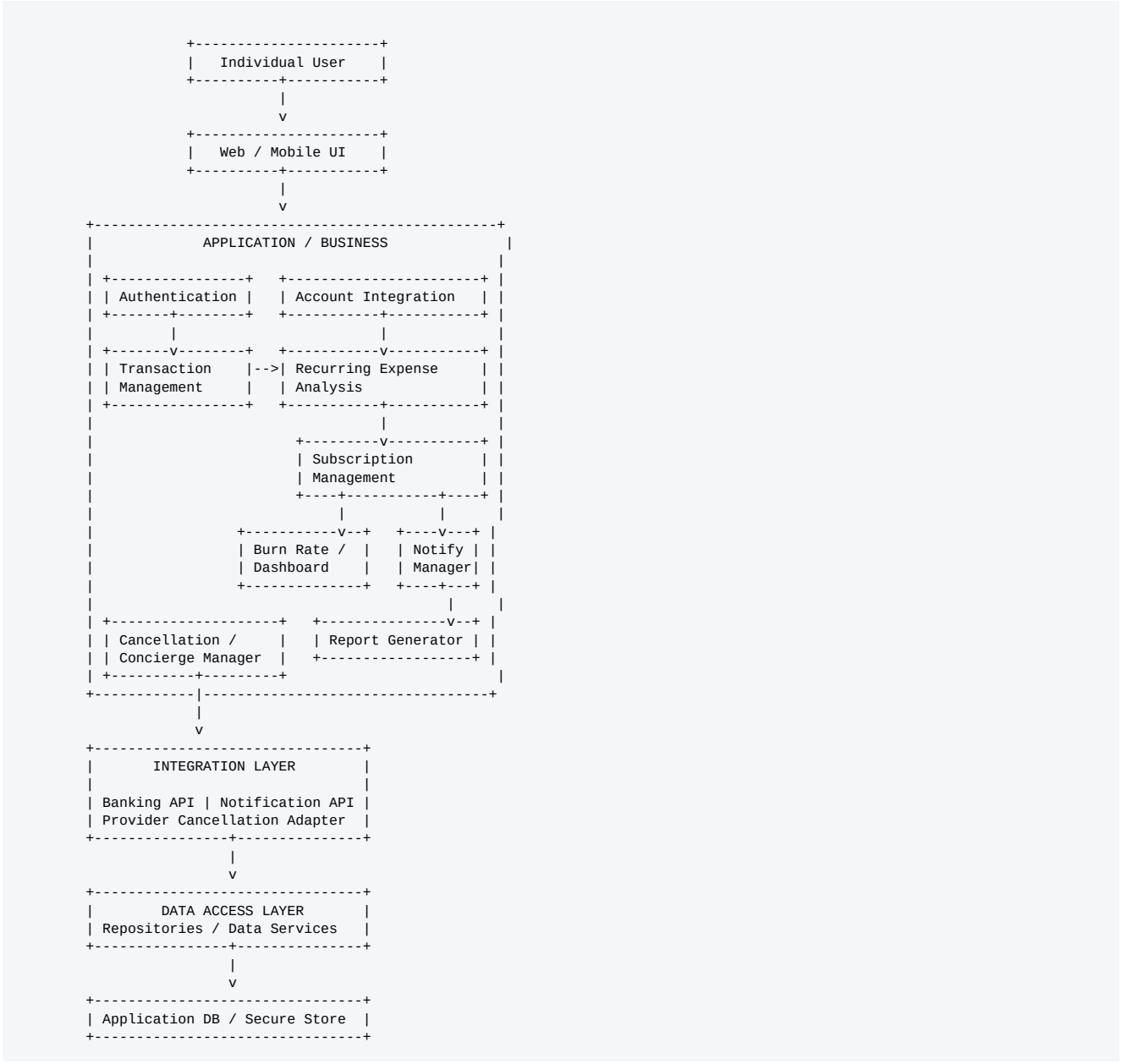
The proposed system uses a layered architecture as its primary architectural pattern. Within the application layer, the business functionality is decomposed into cohesive modules such as transaction management, recurring expense analysis, subscription management, notifications, cancellation, and reporting. External systems are isolated behind integration components, while persistence is accessed through a dedicated data-access layer.

The major layers are:

Layer	Responsibility	Representative Components
Presentation	User interaction and visualization.	Web/Mobile UI, Dashboard
Application / Business	Implements business rules and coordinates use cases.	Auth, Transaction, Recurring Analysis, Subscription, Notification, Cancellation, Reporting
Integration	Communicates with external systems using controlled interfaces.	Banking API Adapter, Provider Cancellation Adapter, Email/Push Adapter
Data Access	Persists and retrieves application data.	Repositories / Data Access Services
Persistence	Stores application data and tokenized integration data.	Application Database, Secure Token Store

3. Component Diagram

The following component model is derived directly from the Lab 1 functional and non-functional requirements. The central business modules are kept cohesive, while banking, notification, and provider cancellation dependencies are isolated behind integration components.



External actors and systems

Individual User: interacts with the dashboard, account connections, subscription details and cancellation workflows.
Finance Auditor: requests anonymized, aggregated monthly reports.
Banking API: supplies transaction/account information through a secure read-only integration.
Notification Provider: delivers push/email renewal alerts.
Subscription Provider / Cancellation Portal: supplies provider-specific cancellation routes; supported providers may additionally expose an API or workflow for concierge cancellation.

4. Component Catalogue

Component	Responsibility	Inputs / Outputs
Authentication Component	Authenticates users and manages application sessions. Keeps authentication concerns separate from financial data processing.	Credentials/session tokens → authenticated application session
Account Integration Component	Connects to a supported banking API using read-only authorization and imports account/transaction information.	Bank API → normalized account and transaction data
Transaction Management	Validates, normalizes, stores and retrieves imported transaction records.	Raw/normalized transactions ↔ database
Recurring Expense Analysis	Analyzes transaction history to detect recurring billing patterns and estimate subscription expenditure.	Transactions → recurring patterns, renewal dates, estimated monthly cost
Subscription Management	Maintains identified subscription records, provider details, status, renewal date and recurring cost.	Analysis results ↔ subscription records
Burn Rate / Dashboard	Aggregates active recurring expenses and presents monthly subscription burn rate and subscription details.	Subscription data → dashboard metrics
Notification Manager	Schedules and triggers renewal alerts at least 72 hours before an identified renewal.	Renewal schedule → push/email notification
Cancellation / Concierge Manager	Provides provider-specific cancellation guides/links and coordinates optional authorized concierge workflows.	Subscription + authorization → cancellation workflow/status
Report Generator	Creates anonymized aggregated monthly burn-rate reports in CSV and PDF formats for the Finance Auditor.	Aggregated metrics → CSV/PDF report
Data Access Layer	Provides repository/data-service interfaces so business components do not depend directly on storage implementation.	Business queries ↔ persistence
Secure Token Store / Database	Stores application records and protected/tokenized integration credentials rather than raw banking passwords.	Persistent application data

5. Component Interfaces and Dependencies

Components communicate through explicit responsibilities and data contracts. The UI should not directly access the database or external banking provider. Business components depend on service/repository interfaces, which reduces coupling and makes external providers replaceable.

Interface	Provider	Consumer	Purpose
AuthService	Authentication	UI / Application modules	Login, session validation, authorization
BankingIntegration	Account Integration	Transaction Management	Read-only transaction/account import
TransactionRepository	Data Access	Transaction Management / Analysis	Persist and query transactions
RecurringAnalysisService	Recurring Analysis	Subscription Management	Detect recurring patterns and renewal dates
SubscriptionRepository	Data Access	Subscription Management	Persist subscription state
NotificationService	Notification Manager	Subscription / Scheduler	Send renewal alerts
CancellationService	Cancellation Manager	UI / Subscription Management	Guides, links, status updates
ProviderCancellationAdapter	Integration Layer	Cancellation Manager	Provider-specific cancellation workflow
ReportService	Report Generator	Finance Auditor / UI	Generate anonymized CSV/PDF reports
MetricsService	Burn Rate / Dashboard	UI / Reporting	Calculate recurring expenditure metrics

6. Primary Data Flow

The normal data path begins with a read-only bank connection. Imported transactions are normalized and persisted, then analyzed for recurring patterns. Identified subscriptions feed the dashboard, notification scheduler, and cancellation workflows.

> Banking API → Account Integration → Transaction Management → Recurring Expense Analysis → Subscription Management → Dashboard / Notifications / Cancellation / Reporting

Data-flow sequence

1. User authorizes a read-only bank connection.
2. Account Integration obtains permitted transaction data through the banking API.
3. Transaction Management validates, normalizes and persists the transaction records.
4. Recurring Expense Analysis identifies repeated merchant/billing patterns and estimates renewal dates and monthly costs.
5. Subscription Management stores identified subscriptions and their current status.
6. Burn Rate / Dashboard aggregates active recurring expenses for presentation.
7. Notification Manager schedules an alert at least 72 hours before a known renewal.
8. Cancellation Manager provides a provider-specific guide/link or starts the optional concierge workflow after explicit authorization.
9. Report Generator uses anonymized aggregates to create CSV/PDF reports for the Finance Auditor.

7. Component Interaction for “Cancel Subscription”

The Cancel Subscription use case from Lab 1 is implemented as a collaboration among the dashboard, subscription management, cancellation, provider-integration and burn-rate components.



Standard cancellation flow

- The user selects an identified subscription from the dashboard.
- Subscription Management retrieves provider, cost and renewal information.
- Cancellation Manager obtains the appropriate provider guide and official cancellation portal link.
- The user completes cancellation externally and confirms completion in the application.
- Subscription Management changes the status to Cancelled.
- Burn Rate / Metrics recalculates projected monthly recurring expenditure.

Concierge extension

For supported providers, the optional Concierge Cancellation path extends the cancellation process. The user explicitly authorizes the application to act on their behalf. The Concierge component then uses an approved provider API, automated email, or support-ticket workflow. The subscription can remain in a pending state until the provider confirms cancellation. Notification Manager communicates progress to the user.

8. Traceability: Lab 1 Requirements to Lab 3 Components

Requirement	Mapped Component(s)	Architectural Support
FR-001	Transaction Management; Recurring Expense Analysis; Subscription Management	Transaction data is normalized before analysis; analysis produces recurring patterns, renewal dates and projected monthly expenditure.
FR-002	Authentication; Account Integration; Data Access	Bank access is isolated behind an integration component and uses read-only authorization.
FR-003	Subscription Management; Notification Manager	Renewal dates are maintained by subscription management and consumed by the notification scheduler.
FR-004	Cancellation / Concierge Manager; Provider Adapter	Provider-specific cancellation information is isolated from the core subscription logic.
FR-005	Report Generator; Data Access	Reporting consumes aggregate data and generates CSV/PDF outputs without exposing individual user credentials.
NFR-001	Authentication; Account Integration; Secure Token Store	Sensitive credentials are not part of ordinary transaction records; external access is represented through protected/tokenized credentials.
NFR-002	Burn Rate / Dashboard; Recurring Analysis; Data Access	Computation is separated from presentation and can be optimized through indexed queries, cached aggregates and efficient analysis.

9. Architectural Justification

Separation of concerns. Presentation, business logic, integrations and persistence have distinct responsibilities. This prevents the dashboard from becoming tightly coupled to database or banking-provider implementation details.

Maintainability. Recurring-expense detection, notification scheduling and cancellation logic can evolve independently. A change to the banking provider can be contained within the Account Integration component.

Security. Financial integrations are isolated from ordinary application logic. The architecture supports tokenized/protected access credentials and avoids storing raw banking passwords in application data or logs, aligning with NFR-001.

Performance. The dashboard does not need to repeatedly process every raw transaction during rendering. Recurring analysis and aggregate metrics can be computed, indexed, cached or incrementally updated, supporting the 2-second target in NFR-002 for up to 10,000 records.

Testability. Components expose clear interfaces. For example, the Recurring Expense Analysis component can be tested with synthetic transaction histories without requiring a live bank connection.

Replaceability. External providers such as banking, notification and subscription-provider integrations are adapters at the boundary. This limits vendor-specific code inside core business components.

Scalability. The modular decomposition provides a path for independently scaling computationally heavier functions such as transaction analysis or report generation if the application grows, without requiring the entire system to be redesigned as microservices.

Consistency with the problem statement. Every major component corresponds to a functional requirement, use case, or non-functional constraint defined in Lab 1, keeping the architectural model traceable to the requirements.

10. Alternative Architectural Patterns Considered

Microservices Architecture: It could separate authentication, transaction analysis, notifications, reporting and cancellation into independently deployed services. However, for the current scope it introduces additional deployment, communication and operational complexity without a demonstrated need for independent service scaling.

Event-Driven Architecture: Events such as TransactionImported, SubscriptionDetected and RenewalApproaching could support asynchronous processing. This is useful for notifications and background analysis, but it is better treated as a complementary communication mechanism rather than the primary architecture for this lab.

> Chosen approach: Layered architecture with modular components provides clear separation of concerns while remaining understandable, testable and practical for the specified system scope.

11. Security Considerations

- Use read-only banking permissions wherever the external banking integration supports them.
- Never store raw bank passwords in the application database or application logs.
- Store integration tokens using protected storage and appropriate encryption/access controls.
- Apply authentication and authorization before exposing financial information.
- Minimize personally identifiable information in auditor reports; reports should contain aggregate/anonymized information.
- Keep external cancellation authorization explicit and auditable for the concierge feature.

12. Conclusion

The proposed component model translates the Lab 1 requirements for the Personal Subscription & Recurring Expense Auditor into a structured software architecture. A layered architecture with modular business components separates user interaction, financial analysis, subscription management, external integrations and persistence. The design directly supports the defined use cases, including bank synchronization, recurring-expense detection, renewal notifications, cancellation assistance and anonymized reporting. It also provides clear architectural mechanisms for the security and performance constraints specified in Lab 1.

End of Lab 3 Submission